Coding Challenge

Task Description:
You are working as a software developer in an agile software team. In the next sprint, you have picked the task of developing a robust, platform-independent C# application to manage orders in an SMT (Surface-Mount Technology) manufacturing environment. At the end of the sprint, you will present your results to the team to facilitate an active discussion about the implementation. Providing an outline of the modelled classes and their architecture would be beneficial.

Requirements:
The application should support CRUD operations for **Orders**, **Boards**, and **Components**. The relationships between the entities are as follows:

- An **Order** can include one or more **Boards**, and a Board can be produced in one or multiple orders.
- A **Board** can contain one or more **Components**, and a Component can be placed on one or more Boards.

The following attributes should be included for each entity:

- **Order:** Name, Description, Order Date
- **Board:** Name, Description, Length, Width
- **Component:** Name, Description, Quantity

You may extend these entities with additional properties as needed to accurately represent the relationships between the objects (e.g., foreign keys, IDs, reference lists).

Required actions:

- The application should allow the creation, editing, searching, and removal of one or more: components, boards and orders.
- The application should allow a download of an order – simulate a download to a production line where the order will be produced.

The application should serialize data to **JSON** for interoperability and persist it using a technology of your choice (e.g., **MS SQL**, **SQLite**, **file storage**).

The solution should include a **logging mechanism** using a commonly adopted framework to capture relevant actions and errors.

Apply object-oriented programming principles (e.g., **SOLID**, **DRY**), use standard **design patterns**, and write a **unit test** for at least one representative method to ensure code quality.

Additional Technical Requirements (Optional):

The following technical requirements are **optional** and can be implemented to demonstrate advanced knowledge, completeness, and best practices:

- **Version Control:**
  Use a **free version control system** such as **GitHub** or **Azure DevOps** to host the project. All code changes should be committed to the remote repository. Upon completion, make the repository accessible for **review** (public or restricted access).
- **CI/CD Pipeline:**
  Set up a **build and release pipeline** using a free service like **GitHub Actions** or **Azure DevOps Pipelines**. The pipeline should:
  - Automatically build the application on each commit.
  - Deploy the application to a **free-tier cloud provider** (e.g., Microsoft Azure).
- **Web Interface:**
  The application should include a **web-based user interface** accessible via login. The interface should:
  - Communicate with a **backend Web API** (e.g., ASP.NET Core Web API).
  - Enable CRUD operations for orders, boards, and components.
  - Include authentication (e.g., custom login or integration with a free identity provider like **Azure AD B2C** or **Firebase Authentication**).
- **Persistence Between Restarts:**
  Choose a persistence mechanism to store application data across restarts:
  - **Database** (e.g., Azure SQL, PostgreSQL, SQLite)
  - **Blob/file storage**
- **Dockerization / Containerization:**
  Containerize the entire solution (API, database, frontend) using **Docker**. Provide a Dockerfile and optionally a docker-compose.yml file to simplify deployment and local development. The containerized app should be runnable both **locally** and in the **cloud**.